# JavaScript Assignment

Event Loop, Promises, async/await & Generators — Tricky Edition

---

This assignment is based on the concepts you covered today: setTimeout / setInterval, microtask vs macrotask ordering, promise chaining, Promise.all / allSettled / race / any, async/await, and generators. It is intentionally tricky — several tasks are designed to break your intuition about execution order and error handling, not just test syntax.

| # | Task | File | Core Trap / Concept |
|---|---|---|---|
| 1 | Predict the Exact Output | eventLoop.js | Sync vs microtask vs macrotask vs setInterval ordering |
| 2 | Fix the Silent Race Condition | chainBug.js | Missing return inside .then() chain |
| 3 | Timeout Race | fetchWithTimeout.js | Promise.race() to timeout a slow request |
| 4 | Partial Failure Report | stationReport.js | Promise.allSettled() vs Promise.all() |
| 5 | First Success Wins | bestMirror.js | Promise.any() when some promises reject |
| 6 | Async Generator Ticket Counter | ticketGen.js | async function* + for await...of |
| 7 | Trick Questions (Written) | answers.js | Conceptual traps, answered as comments |

## Task 1 — Predict the Exact Output

*Maps to: eventLoop.js | Trap: sync code vs microtasks (Promises) vs macrotasks (setTimeout/setInterval)*

```
console.log("A. Station Master starts duty");

setTimeout(() => console.log("B. Late train arrives"), 0);

const announce = setInterval(() => {
  console.log("C. Repeated announcement");
}, 100);

Promise.resolve()
  .then(() => console.log("D. Microtask 1"))
  .then(() => console.log("E. Microtask 2"));

setTimeout(() => {
  console.log("F. Clearing announcements");
  clearInterval(announce);
}, 250);

console.log("G. Station Master ends duty");
```

1. Without running the code, write down the exact console output order for the first 5 lines that get printed (A through G, in the order they actually appear).
2. Then actually run the file in Node and compare — write a one-line comment explaining any line where your prediction was wrong.
3. In a comment, explain WHY "G" prints before "D" and "E", and why "D" and "E" print before "B", even though the Promise has no delay and setTimeout has a 0ms delay.

4 In a comment, explain why "C" keeps printing every ~100ms until "F" runs, and why the interval must be explicitly cleared or the Node process would never exit on its own.

## Task 2 — Fix the Silent Race Condition

*Maps to: chainBug.js | Trap: a .then() that forgets to return a promise*

```
// This code is BUGGY on purpose. It looks like it chains correctly,
// but Station 2 does NOT actually wait for Station 1's delayed work.

function stationOneWork() {
  return new Promise((resolve) => {
    setTimeout(() => resolve("■ Parcel sorted at Station 1"), 1500);
  });
}

function stationTwoWork(data) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`■ Station 2 processed: ${data}`), 500);
  });
}

function runPipeline() {
  return Promise.resolve("■ Parcel dispatched")
    .then((data) => {
      console.log("Step 1:", data);
      stationOneWork(); // BUG: result is never returned or awaited
    })
    .then((data) => {
      console.log("Step 2:", data); // logs "undefined" instead of Station 1's result
      return stationTwoWork(data);
    })
    .then((finalData) => {
      console.log("Step 3:", finalData);
    });
}

runPipeline();
```

1 Run the buggy code above and observe that Step 2 logs **undefined** and the pipeline finishes almost instantly — it does NOT wait 1500ms for Station 1.

2 Fix the bug with the smallest possible change (hint: it's one missing keyword) so that Step 2 correctly logs Station 1's resolved message, and the whole pipeline takes at least 2000ms.

3 Rewrite the fixed runPipeline a second time using async/await instead of .then() chaining, producing identical output and timing.

4 In a comment, explain in your own words why forgetting to return a promise inside a .then() callback breaks the chain silently instead of throwing an error.

## Task 3 — Timeout Race

*Maps to: fetchWithTimeout.js | Trap: Promise.race() where the loser keeps running in the background*

```
function unreliableServer(delayMs, label) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`■ Response from ${label}`), delayMs);
  });
}
```

1. Write a function withTimeout(promise, ms) that returns a new promise which resolves/rejects with whatever promise settles with, UNLESS ms milliseconds pass first — in which case it rejects with new Error("■ Timeout"). Use Promise.race() internally.
2. Call withTimeout(unreliableServer(3000, "Server A"), 1000) and log the result using .catch() — it should reject with the Timeout error after ~1000ms.
3. Call withTimeout(unreliableServer(500, "Server B"), 1000) and log the result — it should resolve with Server B's response after ~500ms.
4. In a comment, explain the trap: even after the timeout wins the race and your code moves on, the original 3000ms setTimeout from Server A is STILL running in the background. Explain why Promise.race() cannot cancel the loser, and name one real API (e.g. AbortController) that is normally used to actually stop it.

## Task 4 — Partial Failure Report

*Maps to: stationReport.js | Trap: Promise.all() short-circuits on the first rejection*

```
function checkStation(name, willFail) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (willFail) reject(new Error(`${name} is OFFLINE`));
      else resolve(`${name} is OK`);
    }, Math.random() * 1000);
  });
}

const stations = [
  checkStation("Chennai", false),
  checkStation("Bangalore", true),
  checkStation("Vijayawada", false),
  checkStation("Nellore", true),
];
```

1. First, call Promise.all(stations) with .then()/.catch() and log what happens. Notice you only ever see ONE error, even though two stations failed.
2. Now solve it properly using Promise.allSettled(stations). Loop through the results and build a summary object: { okCount, failedCount, failedStations: [...] }, where failedStations lists the error messages of only the rejected checks.
3. Log the final summary object.
4. In a comment, state the one-line rule for when to use Promise.all() vs Promise.allSettled().

## Task 5 — First Success Wins

*Maps to: bestMirror.js | Trap: Promise.any() vs Promise.race() when failures are involved*

```
const mirror1 = new Promise((_, reject) => setTimeout(() => reject("Mirror 1 down"), 200));
const mirror2 = new Promise((resolve) => setTimeout(() => resolve("Mirror 2 OK"), 800));
const mirror3 = new Promise((_, reject) => setTimeout(() => reject("Mirror 3 down"), 400));
```

1. Run Promise.race([mirror1, mirror2, mirror3]) with a .catch() attached and log the result. Note that it settles almost immediately with a REJECTION, even though a working mirror exists.
2. Now run Promise.any([mirror1, mirror2, mirror3]) and log the result — it should correctly resolve with Mirror 2's success, ignoring the two rejections along the way.
3. Add a 4th mirror that also fails, so ALL THREE original delays reject and only mirror2 would have succeeded — instead, make all four promises reject on purpose, and wrap Promise.any() in a try/catch (using async/await). Log the caught error and, in a comment, name the special error type Promise.any() throws when every input promise rejects.

## Task 6 — Async Generator Ticket Counter

*Maps to: ticketGen.js | Trap: combining generators with async delays, using for-await-of*

```
// A plain generator only yields instantly. This task asks you to build
// an ASYNC generator, which can yield values that arrive over time.
```

1. Write an **async generator function** issueTickets(count) that, each time it is asked for the next value, waits 500ms (using a Promise + setTimeout inside the generator) and then yields a string like "■ Ticket #1 issued", "■ Ticket #2 issued", up to count tickets.
2. Consume it with a for await (const ticket of issueTickets(4)) loop and log each ticket as it arrives, roughly 500ms apart.
3. Manually call .next() on the async generator (without the for-await-of loop) exactly twice, and log the two returned objects. Since each call returns a Promise, you must await each one.
4. In a comment, explain the difference between what a normal function*'s .next() returns versus what an async function*'s .next() returns.

## Task 7 — Trick Questions (Answer as Comments)

*Maps to: answers.js | No code execution needed — answer each as a comment block*

1. If you chain 3 .then() calls onto an already-resolved promise, and ALSO have a setTimeout(fn, 0) queued before any of them run — which runs first, and why?
2. Does await block the entire JavaScript engine (all other code everywhere) while it waits, or does it only pause the current async function? Justify with one sentence.
3. If a for...of loop is used on a plain (non-async) generator that internally returns promises instead of plain values, what will the loop variable actually be on each iteration — the resolved value, or the Promise object itself? Why?
4. In Promise.allSettled(), what are the exact two possible values of the status field in each result object, and what other field accompanies each one?

---

**Submission note:** Each numbered file above should run independently with node filename.js and print clearly labeled console output for every step. Where a task asks for an explanation, write it as a // comment directly above or below the relevant code — do not just keep it in your head.